CHAPTER 4: AUGMENTED CALL, WORKFLOW, AGENT

Summary: *Three architectural patterns, and one question that picks between them. Most of the difficulty here is that the wrong pattern still works, so the cost of choosing badly shows up as unpredictable spend, unauditable behaviour, or flexibility nobody ever uses rather than as a system that fails. This chapter gives you the selecting question, what each pattern actually costs, why decomposition is a structural move rather than a tuning one, and which prompting technique repairs which defect.*

The One Question That Selects the Pattern

Before comparing patterns, answer this: is the **path through the work settled before any request shows up**?

That single question does most of the work, and almost every wrong answer in this area comes from answering a different one, usually “how complicated is this?” Complexity does not select a pattern. Predictability of the route does.

Three cases, and they map one to one.

The **route is a single step**. One call, given whatever facts or actions it needs from outside. That is an augmented call.

The **route is known and fixed before the work starts**. A sequence somebody could draw in advance, the same sequence every time. That is a workflow.

The **route only becomes clear during the work**, because which steps are needed depends on what the earlier steps found. That is an agent.

The third definition is the precise one and worth holding exactly: an agent **picks each next step** by looking at **what the last one returned**. Not the pattern that is more capable, more advanced or more autonomous. The one where step *n+1* is a function of step *n*’s output rather than of a plan written beforehand.

Three Patterns and What Each One Costs

Pattern	What it is	Choose it when	What it costs

Pattern	What it is	Choose it when	What it costs
Augmented call	One model call, given tools, retrieval or both	The task is one step and the model needs facts or actions it does not hold	Little visibility into how the answer was reached
Workflow	A fixed sequence of calls, each one separately inspectable	The path is settled before any request shows up	Somebody has to define the steps, and steps that never vary still get run
Agent	Each next step selected from the last result	Which steps are needed emerges during the task	Cost per request is not predictable, and behaviour is materially harder to audit

Anthropic's own guidance lines up with that split and adds useful detail. Explicit prompt chaining, meaning breaking a task into sequential API calls, is recommended where intermediate outputs need inspecting or where a pipeline structure has to be enforced, with each step run as a separate call precisely so it can be logged, evaluated or branched on. The most common chaining pattern is self-correction: produce a draft, review it against criteria, then refine. Subagents are recommended for work that can run in parallel, needs isolated context, or forms genuinely independent workstreams. Direct work is recommended for simple tasks, for sequential operations, and for tasks that have to carry context across steps.[1]

Read that last clause carefully, because it cuts against instinct. Work that must **carry context from step to step** is a reason to **keep it in one place**, not a reason to **split** it. Splitting is what costs you the context.

There is a fourth structural option that is easy to miss and often the right answer when a task makes many similar calls. Programmatic tool calling lets the model write code that calls your defined tools from inside a code execution container, instead of taking a model round trip for every single invocation. The documented example collapses 20 separate round trips, one per employee in a budget check, into a single script. Layered on top of basic search tools, it improved performance on agentic search benchmarks by an average of 11%, while using 24% fewer input tokens.[2] When a scenario describes the **same tool being called many times over a list**, the structural fix is usually to **move the loop into code** rather than to **make the agent smarter**.

Flexibility You Pay For and Never Use

The most common pattern error is reaching for an agent because the domain sounds complicated.

Here is the tell. When the scenario describes a process whose steps are stated as fixed and known in advance, an option arguing for adaptability is arguing for something real. Adaptability *is* a genuine reason to choose an agent. It is just not a reason available in this scenario, because nothing in the described process ever varies. You are paying for **runtime planning capacity** that will **never once change a decision**.

The bill arrives in three places. Cost per request stops being predictable, because the number of steps is no longer fixed. Auditing gets harder, because the sequence differs run to run and “what did it do?” becomes a question you answer from logs rather than from the design. And validation gets weaker, because step boundaries are where you check intermediate results, and an agent decides its own boundaries.

That last one is worth inverting, because it is a live reason to keep a workflow even when an agent would work. **Step boundaries are an asset**. They give you somewhere to validate, somewhere to attribute cost, and somewhere to stop.

The **named anti-pattern is multi-agent coordination applied to a known route**. Two agents negotiating over a sequence a human could have drawn on a napkin is overhead with a good story attached.

There is a mirror-image error that gets less attention and costs just as much. When a scenario says the necessity of later steps only becomes apparent partway through, a workflow cannot express that, and no amount of adding branches fixes it. The test between them is whether the **variation could be written down as a branch ahead of time**. If it could, it is a workflow with a condition in it. If it genuinely could not, because it depends on content no human has seen yet, you need an agent.

Where the Facts Live: Weights Against Lookup

One more pattern decision belongs here, and it turns on a rate rather than on a capability.

Whether the model knows the fact today is the wrong thing to measure. What decides this is how quickly the **underlying data moves**, set against how quickly **anything baked into the model can be updated**. Facts that shift in days cannot live somewhere that gets refreshed in months. They belong in a per-request lookup, which means retrieval or a tool call, regardless of how well the model performs on them today.

This is why “the model gets this right in testing” is not an argument for putting a volatile fact in the prompt or trusting recall. Today’s accuracy on last quarter’s data says nothing about next month.

Decomposition Changes the Task, Not the Model

Decomposition gets confused with three things that are not decomposition, and the confusion is worth clearing because the failure signature is distinctive.

When output is incomplete, when the model covers four of the seven required sections, or handles the common cases and silently drops the tail, the instinct is to reach for capacity or guidance: a larger model, a longer instruction, a lower temperature, more examples. All four adjust how well the model performs the task as posed.

Decomposition changes the task as posed. Splitting “review this contract” into an explicit pass per clause type does not make the model better at reviewing contracts. It makes exhaustive coverage structural, because now there is a step per category and a missing one is visible. It also produces something the other three do not: an

inspectable intermediate output, so you can see which pass failed rather than inferring it from a bad final answer.

That is the discriminating property. When a failure signature is **"it did not cover everything"** or **"we cannot see where it went wrong"**, the fix is **structural**. **Capacity, guidance and sampling** do not produce **coverage guarantees or intermediate artifacts**.

Zero-Shot, Few-Shot, and Chain-of-Thought: What Each Repairs

Three techniques, three different defects. Matching them wrongly is the most common error in this objective.

Technique	What it is	What it actually repairs	What it does not repair
Zero-shot	Instructions only, no examples	Nothing by itself; it is the baseline	Format precision, multi-step reasoning
Few-shot (multishot)	One or more complete worked examples	Format, tone and structure	Reasoning the model could not do anyway
Chain-of-thought	Asking for reasoning before the answer	Multi-step reasoning errors	Formatting, and anything on a task with no reasoning in it

Examples are described as among the most reliable ways to steer an output's format, tone and structure.[1] The practical form of that matters more than the label: **show one finished output that is already right**, instead of **writing a description** of one. A worked exemplar pins down every formatting property simultaneously, including the ones nobody thought to write a rule about. Prose leaves each property open to interpretation, and the model interprets.

So when a scenario says the format instructions are detailed, explicit and still not being followed, adding a sixth paragraph of instruction is the wrong move and a single complete example is usually the right one.

Chain-of-thought has a sharper boundary and an important negative result. When a technique helps on **one task and not on another**, the honest conclusion is about the **tasks**, not about the **technique**. A task with genuine intermediate reasoning benefits. A task that is essentially recall, extraction or classification has no reasoning to externalise, so asking for reasoning adds tokens, latency and nothing else. That licenses the sensible operational answer, which is to use it on tasks where it earns its tokens and leave it off the ones where it does not. A per-task decision, not a policy.

The wrong conclusions from a split result are all tempting. That the technique is unreliable. That the prompt needed to be longer. That a bigger model would have fixed both. Each locates the cause somewhere it is not, and each leads to a change that does not help.

When the Client Wants the Wrong Pattern

Occasionally the scenario is not technical. Somebody with authority wants the more impressive architecture, and your analysis says the simple one is correct.

Four responses are available and three of them are wrong. Capitulating gives them a system whose cost they cannot predict. Misrepresenting the recommendation to make it sound like what they asked for is a straightforward integrity failure. Withdrawing protects you and helps nobody.

The response that works does two things at once. **Argue the choice on the criteria the client themselves stated**, cost predictability, auditability, time to delivery, whatever they said mattered, rather than on your architectural preference. And **document the conditions under which the answer would change**, so that a path to the agentic design exists if real variability shows up. That keeps the honest recommendation and the relationship, and it converts a disagreement about taste into a disagreement about evidence, which is the only kind that resolves.

How Wrong Answers Are Built on Pattern Selection

- **Paying for variability the scenario does not** have. The option argues for adaptability, autonomy or coordination in a process the stem has already described as fixed. The argument is valid in general and unavailable here. The test: does the scenario say, anywhere, that the **route varies**?
- **Reaching for capacity when the defect is structural**. The option offers a larger model, longer instructions, lower temperature or more examples against a failure of coverage or inspectability. The test: does this change **make the task different**, or just **make the model try harder** at the same task?
- **Matching the technique to the wrong defect**. Chain-of-thought against a formatting problem, examples against a reasoning problem, either against a task with no reasoning to do. The test: **name the defect first**, then pick the technique.
- **Blaming the technique for a task-shaped result**. The option concludes that a method is unreliable, or that the prompt was too short, when the same method worked elsewhere. The test: what **differs between the two tasks**?

When the scenario says	The answer is usually	The trap is usually
The steps are fixed and settled before work starts	A workflow, with the step boundaries kept for validation and cost predictability	An agent, argued for on flexibility that this process never exercises
Which steps are needed becomes clear only mid-task	An agent	A workflow with more branches, which cannot express variation nobody can enumerate

When the scenario says	The answer is usually	The trap is usually
The variation could be written down as a condition in advance	A workflow with a branch	Treating any variation at all as evidence that you need autonomy
The same tool gets called many times across a list	Move the loop into code that calls the tools directly	Making the agent smarter, or accepting a round trip per item
Intermediate outputs need inspecting, or a pipeline must be enforced	Explicit chaining, each step a separate call	One large call that produces the final answer and no visible middle
Context has to carry from step to step	Keep it in one place	Splitting into subagents, which is what loses the context
The underlying facts change in days or weeks	Per-request lookup, retrieval or a tool	Relying on model knowledge because it tested well
Output is incomplete or you cannot see where it went wrong	Restructure the task so coverage is enforced and the middle is visible	A larger model, a longer prompt, or a lower temperature
Format instructions are explicit and still not obeyed	One complete worked example	More prose describing the format
A technique helped on one task and not another	The difference is in the tasks	Concluding the technique is unreliable, or that the prompt was too short

When the scenario says	The answer is usually	The trap is usually
A stakeholder wants the more elaborate architecture	Argue it on their own stated criteria, and write down what would change the answer	Agreeing, softening the recommendation, or refusing the work

The habit that carries: before choosing a pattern, say out loud **what varies between two requests**. If nothing varies, the route is fixed and the answer is a workflow, no matter how sophisticated the domain sounds.

What to Remember

- What selects a pattern is whether the **path through the work is settled** before any request shows up. Not how complex the domain is. **One step is an augmented call**, a **settled path is a workflow**, and a **path that only emerges** during the work is an agent.
- An agent is precisely the pattern that **picks each next step** by looking at **what the last one returned**. If the variation could have been written down in advance as a branch, it is a **workflow with a condition** in it, not an agent.
- Agentic flexibility costs **predictable per-request spend**, **auditability**, and **step-level validation**. Choosing it for a fixed process buys planning capacity that never changes a decision. **Step boundaries are an asset**: they are where you validate, attribute cost, and stop.
- **Explicit chaining** is recommended where intermediate outputs need inspecting or a pipeline must be enforced, each step a **separate call** so it can be logged, evaluated or branched on, with **self-correction (draft, review against criteria, refine**) as the most common pattern. Subagents suit **parallel work**, **isolated context** and **independent workstreams**. Work that must **carry context across steps** should not be split, because splitting is what loses the context.
- **Programmatic tool calling** moves the loop into code, replacing a model round trip per invocation, and is documented as collapsing **20 round trips**

into one script, improving agentic search benchmarks by an average of **11**% while using **24% fewer input tokens**.

- Facts that move faster than anything baked into the model can be updated belong in a **per-request lookup**, not in **weights or recall**, however well the model handles them in testing today.
- **Decomposition changes the structure of the task**; a bigger model, a longer prompt, more examples and a lower temperature all change how **hard the model tries at the same task**. When the failure is **incomplete coverage or an invisible middle**, only the structural change produces coverage guarantees and inspectable intermediate output.
- **Few-shot repairs format, tone and structure**; **chain-of-thought repairs multi-step reasoning**. For a format defect, hand over **one finished output that is already right** rather than describing what right looks like, because an exemplar fixes every property at once and prose leaves each open to interpretation.
- A task with **no reasoning** in it gains nothing from chain-of-thought. When a technique helps on one task and not another, the **difference is in the tasks**, so use it where it earns its tokens and leave it off elsewhere.
- When a stakeholder wants the wrong pattern, **argue it on the criteria they stated** and **record what would change the answer**. Capitulating, misrepresenting the recommendation and withdrawing are all failures.